FH University of Applied Sciences
TECHNIKUM WIEN

MASTER'S THESIS

Thesis submitted in fulfillment of the requirements for the degree of Master of Science in Engineering at the University of Applied Sciences Technikum Wien - Degree Program Information Systems Management

Retrieval-Augmented Generation for Cyber Threat Intelligence: Designing and Evaluating a Prototype to Support Security Analysts

By: Joben Preet Bhinder, BSc.

Student Number: 2410302067

Supervisors: Wolfgang Rogner, BSc. MSc.
Tamás Horváth, BSc. MSc.

Vienna, August 9, 2026

Declaration

“As author and creator of this work to hand, I confirm with my signature knowledge of the relevant copyright regulations governed by higher education acts (see Urheberrechtsgesetz /Austrian copyright law as amended as well as the Statute on Studies Act Provisions / Examination Regulations of the UAS Technikum Wien as amended).

I hereby declare that I completed the present work independently and that any ideas, whether written by others or by myself, have been fully sourced and referenced. I am aware of any consequences I may face on the part of the degree program director if there should be evidence of missing autonomy and independence or evidence of any intent to fraudulently achieve a pass mark for this work (see Statute on Studies Act Provisions / Examination Regulations of the UAS Technikum Wien as amended).

I further declare that up to this date I have not published the work to hand nor have I presented it to another examination board in the same or similar form. I affirm that the version submitted matches the version in the upload tool.“

Vienna, August 9, 2026 Digital Signature

Kurzfassung

tbd

Schlagworte: Cyber Threat Intelligence (CTI), Retrieval-Augmented Generation (RAG), Large Language Models (LLMs), Hybride Retrieval-Verfahren, Unterstützung von IT-Sicherheitsanalysten

Abstract

tbd

Keywords: Cyber Threat Intelligence (CTI), Retrieval-Augmented Generation (RAG), Large Language Models (LLMs), Hybrid Retrieval, Security Analyst Support

Acknowledgements

tbd

Contents

1 Introduction

1.1 Motivation and problem statement

Cyber Threat Intelligence (CTI) turns raw threat data into assessments that security teams can act on. The underlying data is heterogeneous: structured feeds such as CVE/NVD and MISP coexist with unstructured advisories, vendor reports, and security blogs (Sun et al. 2023). In threat hunting, analysts correlate new indicators and attacker techniques across these sources under time pressure, while the volume and change rate of threat information keep growing (Rajapaksha, Rani, and Karafili 2024). A substantial share of analyst time is therefore spent reading and cross-referencing sources rather than investigating.

Large Language Models (LLMs) can summarize and extract information from such material and are increasingly applied in security operations (Divakaran and Peddinti 2024). Three limitations restrict their direct use in CTI. First, their parametric knowledge is static; a deployed model misses vulnerabilities disclosed after training (Gao et al. 2024). Second, LLMs hallucinate: they produce fluent statements that no source supports (Gao et al. 2024). Third, their output carries no provenance, so analysts must verify each claim manually before acting on it (Rajapaksha, Rani, and Karafili 2024).

Retrieval-Augmented Generation (RAG) mitigates these limitations by retrieving evidence from an external, continuously updatable knowledge store at query time and conditioning generation on the retrieved context (Lewis et al. 2020). In the CTI domain, RAG has been applied to individual tasks such as TTP annotation and cyber-attack investigation (Fayyazi, Taghdimi, and Yang 2024; Rajapaksha, Rani, and Karafili 2024). Empirical evidence on how RAG performs across analyst workflows, and which architectural choices drive that performance, remains limited (Gao et al. 2024). This thesis addresses that gap with a working prototype and its empirical evaluation.

1.2 Solution approach

This thesis designs, implements, and evaluates a RAG prototype for CTI analyst support. The prototype ingests representative CTI sources, including CVE/NVD entries, CISA advisories, MISP events, and vendor reports, into a unified index. A hybrid retrieval layer combines lexical

search (BM25) with dense vector search and reranks the merged candidates. Answer generation is layered: fields that must not be wrong, such as severity scores and exploitation status, are rendered deterministically from the metadata of the retrieved passages and cannot be hallucinated, while a locally deployed open-source LLM contributes only the narrative, grounded in the retrieved context through inline citations. Each claim cites the source it draws on, so analysts can verify individual claims instead of trusting the output as a whole.

The evaluation scores answer quality with the RAGAS framework (Es et al. 2023), task-specific completeness with a field-coverage measure, and analyst-oriented quality with a four-dimension rubric. Retrieval variants are compared against each other and against a non-retrieval baseline on a fixed query set covering four representative analyst task types: vulnerability analysis, IOC enrichment, TTP correlation, and cross-source comparison. The empirical findings are consolidated into a reference model for integrating RAG into CTI and threat-hunting workflows.

1.3 Research questions

From this problem context, the following main research question guides the thesis:

Main research question (MRQ)

How can a Retrieval-Augmented Generation (RAG) system leveraging Large Language Models improve the quality, reliability, and operational usefulness of Cyber Threat Intelligence (CTI) analysis and Threat-Hunting workflows?

Three sub-questions decompose the main question along the dimensions the thesis evaluates: measurable retrieval and answer quality (SRQ1), practical utility for analyst tasks (SRQ2), and the transfer of the empirical findings into design principles (SRQ3).

SRQ1 — Information retrieval and answer quality

To what extent does retrieval augmentation improve the completeness, factual accuracy, and recency of CTI information generated by an LLM compared to a non-retrieval baseline?

SRQ2 — Practical utility and workflow support

Which types of CTI analysis tasks can be effectively supported by a RAG-based LLM system, and how well do the generated outputs align with the informational needs of threat-hunting workflows?

SRQ3 — Conceptual design

How can empirical insights from the RAG-LLM prototype be translated into actionable design principles for integrating generative AI into CTI and threat-hunting workflows?

Section 4.7 answers all four questions on the basis of the evaluation results.

1.4 Objectives, scope and delimitation

The objective of this thesis is a functional, empirically evaluated RAG prototype for CTI analyst support, together with a reference model that generalizes the evaluation results. The target group comprises security analysts and threat hunters; security managers who oversee the adoption of AI-assisted tooling in Security Operations Centers (SOCs) are a secondary audience.

Four delimitations bound the scope. First, the thesis does not pursue large-scale data collection; a curated set of representative sources serves as input, because the research focus lies on retrieval and reasoning quality rather than source coverage. Second, the prototype uses open-source components only. Third, the agent-based extension via the Model Context Protocol (MCP) is exploratory and not part of the core evaluation. Fourth, the evaluation combines quantitative metrics with scenario-based analysis; a user study with practicing analysts is out of scope.

1.5 Methodological approach

The thesis proceeds in six phases (Figure 1):

1. **Literature review:** analysis of LLM use in security operations, RAG architectures, and evaluation methods; the resulting findings and gaps are summarized in Chapter 2.
2. **Process analysis:** definition of representative CTI tasks and derivation of the evaluation query set (Section 3.8.1).
3. **Prototype development:** implementation of ingestion, hybrid retrieval, and grounded answer generation (Chapter 3).
4. **Quantitative evaluation:** RAGAS, field-coverage, and rubric scores across retrieval variants and a non-retrieval baseline, including an ablation study.
5. **Qualitative evaluation:** scenario-based analysis of system outputs for the defined CTI tasks.
6. **Reference modeling:** consolidation of the empirical findings into an integration model for CTI workflows (Section 4.6).

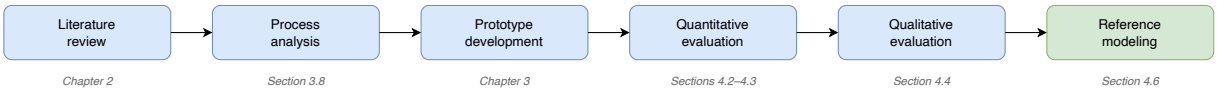

Figure 1: Methodological approach of this thesis

2 Background and related work

2.1 CTI and RAG foundations

Cyber Threat Intelligence (CTI) is evidence-based knowledge about existing or emerging threats, including context, indicators, and actionable advice that informs security decisions (Sun et al. 2023). In practice, CTI is communicated through recurring artifact types. Indicators of Compromise (IOCs) are forensic artifacts, such as file hashes, IP addresses, or domains, that indicate a compromised system (Palacín 2021). Tactics, Techniques, and Procedures (TTPs) describe adversary behavior on three levels: the tactical goal, the method used to achieve it, and its concrete implementation (Ismail et al. 2025). The MITRE ATT&CK framework is the industry standard for categorizing these behaviors (Lekssays et al. 2025). Threat hunting builds on CTI: analysts proactively and iteratively search their environment for signs of compromise that automated detection has missed, typically starting from intelligence about the adversary (Palacín 2021; Sun et al. 2023).

The as-is analyst workflow follows a recurring cycle from planning and collection through processing and analysis to dissemination (Palacín 2021). Across this cycle, the literature reports three bottlenecks, which Figure 2 locates in the workflow:

1. **B1 — Information overload:** the volume, heterogeneity, and informal structure of threat feeds and reports exceed what analysts can process consistently (Alhuzali 2025; Rastogi and Alam 2023).
2. **B2 — Recency:** threat knowledge changes faster than static knowledge bases, including the parametric knowledge of trained language models, can follow (Paul et al. 2025; Chen et al. 2023).
3. **B3 — Verification effort:** extracting information from unstructured reports and validating it remains manual expert work, and outputs without transparent provenance cannot be acted on (Rajapaksha, Rani, and Karafili 2024; Tseng et al. 2024).

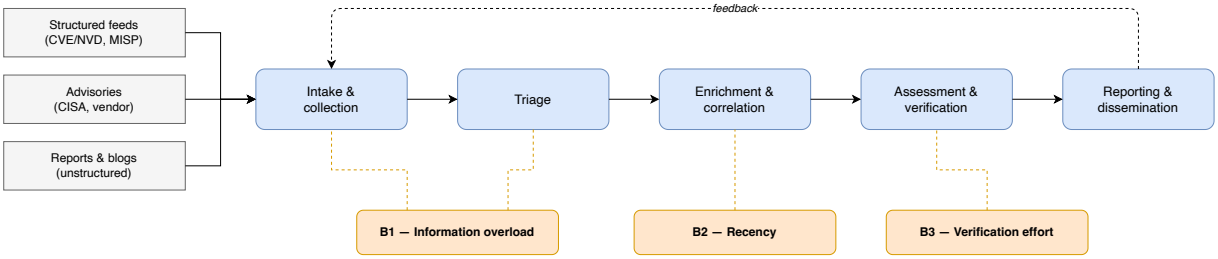

Figure 2: Generic CTI analyst workflow with bottlenecks B1–B3

Retrieval-Augmented Generation (RAG) grounds LLM output in external knowledge. A RAG pipeline indexes documents by cleaning, chunking, and embedding them, retrieves the passages most relevant to a query, optionally reranks them, and assembles them into the prompt from which the model generates its answer (Figure 3) (Gao et al. 2024). This design targets the LLM limitations most relevant to CTI: hallucination, outdated knowledge, and reasoning that cannot be traced to sources (Gao et al. 2024; Chen et al. 2023).

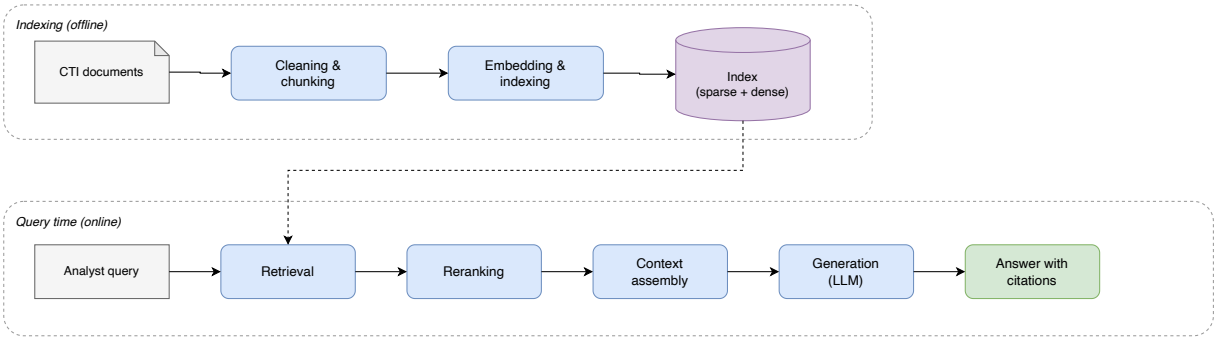

Figure 3: Generic RAG pipeline: offline indexing and query-time stages

Retrieval can be sparse, dense, or hybrid; Table 1 contrasts the strategies and the role of reranking (Agnihotram and Sarkar 2025).

Strategy	Principle	Strength	Limitation
Sparse (BM25)	exact term matching with term-frequency weighting	efficient; reliable for explicit identifiers	misses semantically equivalent formulations
Dense	embeddings in a shared vector space	captures paraphrases and related concepts	can surface related yet irrelevant passages
Hybrid	parallel sparse and dense retrieval, merged by rank or score fusion	keeps exact matches while adding semantic coverage	merged list still needs reordering
Reranking	cross-encoder scores each query–passage pair	raises precision with little recall loss	adds latency

Table 1: Retrieval strategies and reranking in RAG pipelines

These trade-offs matter for CTI text, where the same threat actor appears under aliases such as APT28 and Fancy Bear and distinct ATT&CK techniques share overlapping keywords; hybrid retrieval with reranking therefore suits CTI corpora (Alhuzali 2025; Lekssays et al. 2025). Citing the retrieved sources in the generated answer makes it verifiable: analysts can inspect the evidence behind each claim instead of trusting the model (Rajapaksha, Rani, and Karafili 2024; Liu et al. 2023).

2.2 Key findings from the literature

Table 2 summarizes the key findings of the literature review in four topic areas; the research gaps in Section 2.3 follow from them.

Topic area	Key finding	Source
CTI analyst workflows	The volume and informal structure of threat feeds overwhelm analysts; full automation is not feasible	(Rastogi and Alam 2023; Alhuzali 2025)
	Analyzing natural-language CTI reports is labor-intensive and repetitive and delays response	(Tseng et al. 2024; Rajapaksha, Rani, and Karafili 2024)
LLMs in cybersecurity	LLMs support vulnerability analysis, summarization, and IOC/TTP extraction; junior analysts gain the most	(Divakaran and Peddinti 2024)
	Recurring failure modes are hallucination, outdated knowledge, and technical misinterpretation	(Alhuzali 2025)

continued on next page

Topic area	Key finding	Source
	LLMs over-trust retrieved context and can be misled by incorrect information in it	(Chen et al. 2023)
RAG for CTI	Hybrid retrieval with reranking addresses terminology variance (actor aliases) and overlapping keywords in security text	(Alhuzali 2025; Lekssays et al. 2025)
	Retrieval grounding raises precision in TTP mapping, where decoder-only LLMs show high recall but low precision	(Fayyazi, Taghdimi, and Yang 2024)
	Existing CTI-RAG prototypes each target a single task, e.g. attack investigation, technique annotation, or event response	(Alhuzali 2025; Lekssays et al. 2025; Ismail et al. 2025)
RAG evaluation	Retrieval is assessed with Precision@k and Recall@k plus rank-aware metrics such as MRR and nDCG	(Agnihotram and Sarkar 2025; Yu et al. 2024)
	Reference-free frameworks (RAGAS, ARES) score faithfulness, answer relevancy, and context relevance without gold answers	(Es et al. 2023; Saad-Falcon et al. 2024)
	LLM-as-a-judge scoring risks reinforcing model biases and can diverge from human judgment in specialized domains	(Akkiraju et al. 2024; Yu et al. 2024)
	Ground-truth datasets for CTI tasks are scarce; few public benchmarks (e.g. CTIBench) exist	(Xu et al. 2025; Alhuzali 2025)

Table 2: Key findings of the literature review

2.3 Research gaps

The findings in Table 2 leave three research gaps open:

1. **G1 — Workflow-level empirical evidence:** Existing CTI-RAG prototypes each address a single task, such as attack investigation or technique annotation. Whether these results carry over to the range of tasks an analyst works through, and which architectural choices matter for that transfer, remains empirically open.
2. **G2 — Evaluation realism:** Reported evaluations center on controlled benchmarks and isolated technical metrics, and ground-truth datasets for CTI tasks are scarce. Evidence that connects retrieval and generation quality to representative analyst tasks is rare.
3. **G3 — Source reliability:** RAG pipelines can ingest and propagate misinformation, and

LLMs tend to over-trust retrieved context. How CTI-RAG systems should account for the integrity and reliability of their sources is not settled.

These gaps, together with the workflow bottlenecks B1–B3 from Section 2.1, define the design space of this thesis: Section 3.2 derives design criteria from them, and Section 4.5 assesses the prototype against these criteria.

3 Solution: design and implementation

3.1 Task formulation

The task addressed in this thesis is grounded question answering over CTI sources. The prototype operates on a corpus $\mathcal{D}$ of documents collected from heterogeneous CTI sources and segmented into a set of passages $\mathcal{P}$ (Section 3.4). Given an analyst query q, a retrieval function R selects the k highest-ranked passages as context:

$$\mathcal{C}_k = R(q, \mathcal{P}), \qquad |\mathcal{C}_k| = k. \tag{1}$$

A generation model g then produces the answer

$$a = g(q, \mathcal{C}_k), \tag{2}$$

subject to a grounding constraint: every factual claim in a must be attributable to at least one passage in $\mathcal{C}_k$ and marked with an inline citation to that passage. The analyst can therefore verify each claim against its source instead of trusting the answer as a whole (bottleneck B3, Section 2.1).

In the hybrid configuration, R combines a lexical relevance score (BM25) with a semantic similarity score (dense embeddings) and reranks the fused candidates; Section 3.5 describes this instantiation. Setting $\mathcal{C}_k = \emptyset$ yields the non-retrieval baseline

$$a_0 = g(q, \emptyset), \tag{3}$$

in which the model answers from parametric knowledge alone. This baseline corresponds to configuration V0 in the evaluation (Section 3.8.3); the comparison between a and a_0 quantifies the effect of retrieval augmentation and thereby operationalizes SRQ1.

3.2 Requirements and design criteria

The design of the prototype follows six criteria. They are derived from the workflow bottlenecks B1–B3 (Section 2.1), the research gaps G1–G3 (Section 2.3), and the scope set in Section 1.4, which restricts the prototype to open-source components and local deployment. Table 3 lists the criteria with their origin and the research question they serve; Section 4.5 assesses the prototype against them.

ID	Design criterion	Origin	Ref.
C1	Evidence-linked outputs: every key claim carries an explicit citation of the retrieved source	B3, G1	SRQ1
C2	Hybrid retrieval: combined lexical (BM25) and semantic retrieval with reranking	B1, G1	SRQ1
C3	Source integrity: ingestion restricted to curated, authoritative sources with traceable source metadata	G3	MRQ
C4	Scenario-based evaluation: assessment on representative CTI analyst tasks beyond offline metrics	G2	SRQ2
C5	Analyst-oriented presentation: outputs structured along task coverage, clarity, and transparency	B1, G2	SRQ2
C6	Open-source, locally deployable stack	Scope	MRQ

Table 3: Design criteria with traceability to bottlenecks, research gaps, and research questions

C1 and C2 respond to the verification and overload bottlenecks. Outputs without provenance cannot be acted on (B3), so every generated claim must be attributable to a cited source (C1); this implements the grounding constraint of the task formulation (Equation 2). CTI text combines exact identifiers such as CVE numbers with strongly varying terminology (B1), which single-strategy retrieval serves poorly (Section 2.1); retrieval therefore combines lexical and semantic search and reranks the merged candidates (C2).

C3 responds to G3: because RAG pipelines can propagate manipulated or low-quality content, ingestion is limited to a curated set of authoritative CTI sources, and every indexed passage keeps metadata that identifies its origin. The prototype does not score source credibility at query time; this boundary is revisited in Section 5.2.

C4 and C5 constrain the evaluation and the presentation layer. Offline metrics alone say little about operational usefulness (G2), so the evaluation includes representative analyst scenarios (C4), and answers are structured to match analyst needs rather than optimized for generic quality scores (C5).

C6 fixes the deployment constraint from the scope: an open-source, locally deployable stack keeps sensitive analyst queries on premises, avoids licensing dependencies, and makes the

experiments reproducible.

3.3 System architecture

Figure 4 shows the prototype as a container view following the C4 model (Brown 2018). Two paths separate building the system from using it: an offline path ingests sources and builds the retrieval artifacts, and an online path serves analyst queries. The architecture instantiates the generic RAG pipeline of Figure 3. In the figure, the numbered markers trace an analyst query through the online path, and the circled letters locate the notation of Section 3.1 in the system.

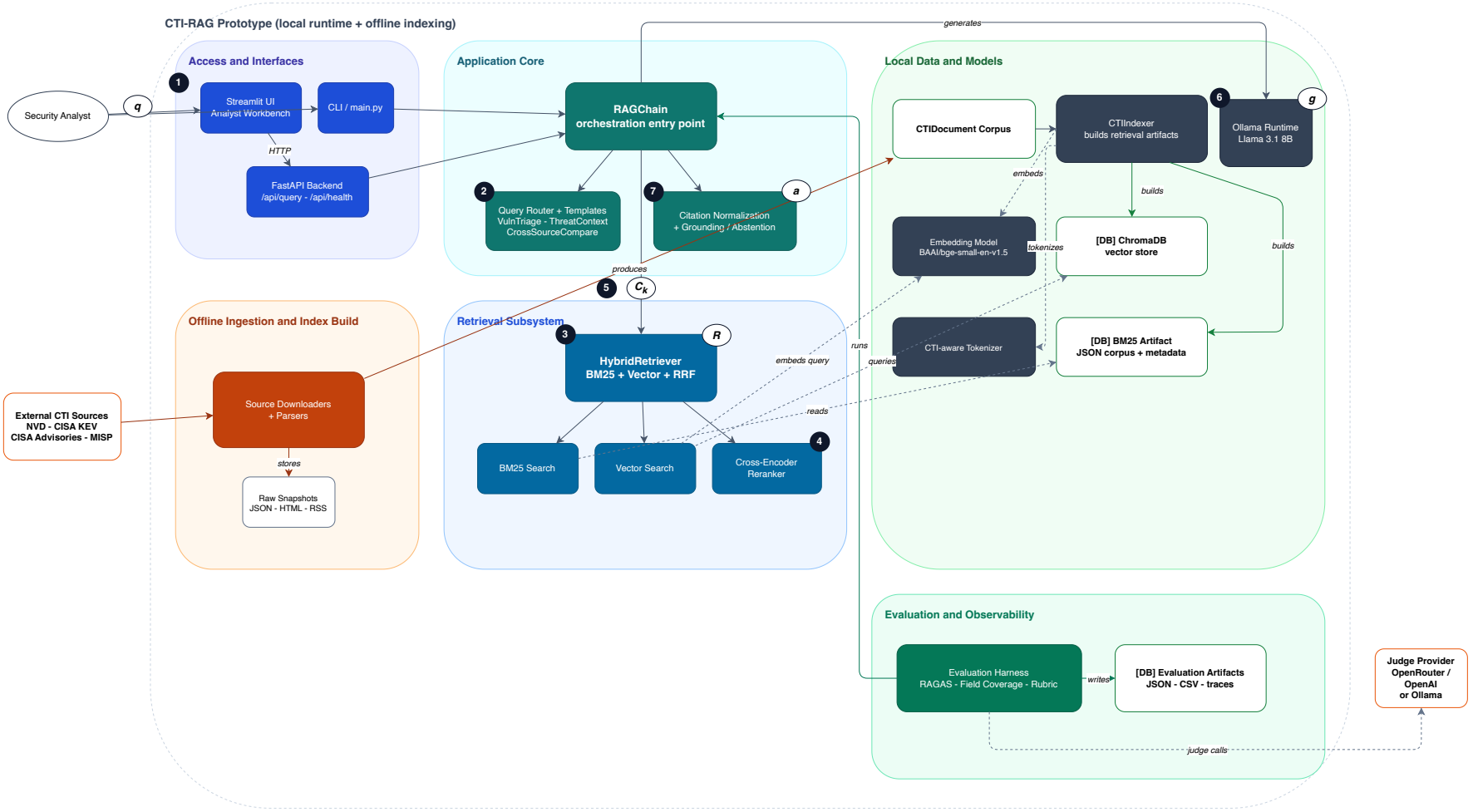

Figure 4: System architecture of the prototype (C4 container view). Numbered markers trace the online query path; circled letters locate the notation of Section 3.1.

On the online path, the analyst submits a query q through the Streamlit workbench, which calls a FastAPI backend, or directly through the command line (step 1). The RAGChain component orchestrates each request: a query router selects one of three task templates (vulnerability triage, threat context, cross-source comparison), onto which the four task types of the evaluation map (step 2, Section 3.8.1). The HybridRetriever implements R: it runs BM25 search over a JSON artifact using a CTI-aware tokenizer and vector search over the ChromaDB store (step 3), merges both rankings by reciprocal rank fusion, and reranks the merged candidates with a cross-encoder (step 4, C2). The resulting context $\mathcal{C}_k$ (step 5) is passed to the generation model g, a Llama 3.1 8B Instruct model served by the local Ollama runtime (step 6). A citation normalization step maps generated claims to their source passages and enforces grounding: when the retrieved evidence does not support an answer, the prototype abstains instead of generating one (step 7, C1). All models and stores run locally (C6).

The offline path keeps the corpus curated (C3): source downloaders and parsers pull the four CTI sources (NVD, CISA KEV, CISA advisories, MISP), store raw snapshots, and produce the document corpus $\mathcal{D}$, from which the CTIIndexer builds the two retrieval artifacts with per-passage source metadata. The evaluation harness drives the same RAGChain that serves analysts, so measured behavior equals operational behavior (Section 3.8).

3.4 Data ingestion and preprocessing

The corpus combines four curated CTI sources (C3): CVE entries from the NVD, restricted to high and critical severity (CVSS ≥ 7.0) and the years 2020–2025; the CISA Known Exploited Vulnerabilities (KEV) catalog; CISA security advisories; and MISP threat-sharing events. The corpus is a fixed snapshot for reproducibility: entries published up to 2025-12-31, fetched on 2026-03-28. Source downloaders store the raw material (JSON, HTML, RSS) unchanged, and source-specific parsers normalize it into a unified document model.

Preprocessing follows a CTI-aware chunking strategy: one CTI artifact, such as one CVE entry or one KEV record, becomes one passage, so that identifiers, scores, and remediation facts stay together. Passages are capped at 512 tokens; only oversized documents are split with a 50-token overlap. Table 4 shows the resulting corpus: $\mathcal{D}$ contains 84,982 documents, and because most CTI artifacts fit into a single passage, $\mathcal{P}$ contains 84,991 passages — only nine advisories required splitting. Every passage carries structured metadata, including source, title, severity, CVSS score and vector, CVE and CWE identifiers, affected products, and publication and modification dates. This metadata preserves provenance for citation (C1, C3) and supports exact-identifier matching during retrieval (Section 3.5).

Source	Content	Documents	Passages
NVD	CVE entries (CVSS ≥ 7.0, 2020–2025)	82,282	82,282
CISA KEV	known exploited vulnerabilities	1,484	1,484
CISA advisories	security advisories	1,028	1,037
MISP	threat-sharing events	188	188
Total		84,982	84,991

Table 4: Corpus statistics of the evaluated index (snapshot: published $\leq$ 2025-12-31, fetched 2026-03-28)

3.5 Hybrid retrieval and reranking

The retrieval function R from Equation 1 runs two retrievers in parallel over the same passage set $\mathcal{P}$. Sparse retrieval uses BM25 with a CTI-aware tokenizer that preserves hyphenated identifiers such as `CVE-2021-44228` or `CWE-79` as single tokens, so that exact identifier matches are not destroyed by standard tokenization. Dense retrieval embeds the query with BGE-small-en-v1.5 and searches the ChromaDB store by cosine similarity. Each retriever returns its 20 highest-ranked candidates.

The two rankings are merged by reciprocal rank fusion. Rank-based fusion avoids calibrating lexical and semantic scores against each other, which have incompatible scales. A cross-encoder then scores each query–passage pair in the fused pool and selects the final context: with the reranker's $k = 5$, this yields the context $\mathcal{C}_k$ of the task formulation. After reranking, a small exact-identifier bonus keeps passages that literally contain a queried CTI identifier visible without dominating the reranker's relevance judgment. This pipeline implements C2; Table 5 lists the configuration.

Stage	Model / method	Parameters
Sparse retrieval	BM25, CTI-aware tokenizer	top-k = 20
Dense retrieval	BGE-small-en-v1.5 (384 dim.)	cosine similarity, top-k = 20
Fusion	reciprocal rank fusion	k_{RRF} = 60, pool size 20
Reranking	cross-encoder ms-marco-MiniLM-L-6-v2	top-k = 5 (final context)
Entity boost	exact-identifier bonus	weight 1.0, applied after reranking

Table 5: Retrieval pipeline configuration (from the frozen `evaluated-v1` state)

3.6 Context assembly, answer generation and citation

The five passages of $\mathcal{C}_k$ are assembled into the prompt as numbered blocks, each exposing a citation label, the source type, the title, and the content. Citation labels are the plain document identifiers, such as `nvd_CVE-2024-3094`; longer composite labels proved impractical because the 8B model could not reproduce them verbatim, which caused valid citations to be lost in normalization.

Generation runs in one of two modes. The *legacy* mode sends a single prompt whose system instruction enforces grounding: the model may use only the retrieved context, must end every claim with an inline citation label, must not invent labels, and must answer in a fixed section structure (summary, relevance, mitigations, evidence, gaps). Listing 3.1 shows the core of this instruction; the complete templates are reproduced in Appendix B. The *templated* mode, the main design contribution of the prototype, splits the answer into layers. Layer 1 is rendered deterministically from the structured metadata of the retrieved passages (CVSS scores, CWE identifiers, KEV dates, triage signals) without any LLM involvement, so these fields cannot be hallucinated. The LLM contributes only the Layer-2 narrative (exploitation context, mitigations), grounded in the retrieved passages via inline citations (C1, C5). Which template is applied follows the query router (Section 3.3).

Two safeguards enforce the grounding constraint at run time. Before generation, a context-support check abstains with an explicit analyst-facing message when retrieval returns no usable evidence, implementing the abstention behavior required by the grounding constraint of Equation 2. After generation, a citation normalization step resolves every cited label against the passages that were actually retrieved, repairing near-miss labels and discarding lines whose citations cannot be resolved. The non-retrieval baseline (V0) uses the same section structure and a knowledge-cutoff instruction matching the corpus snapshot, but no citations — keeping the comparison with the grounded configurations fair.

```
You are a Cyber Threat Intelligence (CTI) analyst assistant. Your role
is to help security analysts by providing accurate, source-grounded
intelligence based only on the retrieved context.

Rules:
1. Use ONLY the retrieved context. Do not use prior knowledge or fill
   gaps from memory.
2. If the context is insufficient, say so explicitly and name what is
   missing.
3. Every factual statement, recommendation, or interpretation that
   comes from the context MUST end with an inline citation using the
   exact Citation Label shown for that chunk. Example:
   "Apply updates per vendor instructions [cisa_kev_CVE-2024-1709]."
4. Never invent a citation label. Only use Citation Labels that appear
   verbatim in the retrieved context.
```

```
[...]
10. In Recommended actions / Mitigations, include only actions
    explicitly supported by the context. If none are supported, write
    exactly: "No reliable mitigation guidance is present in the
    retrieved context."
```

Listing 3.1: System prompt with citation instruction (excerpt; full version in Appendix B)

3.7 Implementation

The prototype is implemented in Python 3.12 as a single repository (Appendix A). LangChain 0.3 provides the orchestration layer with integrations for Ollama, Hugging Face models, and ChromaDB; `rank-bm25` implements the sparse index, `sentence-transformers` serves the embedding and reranking models, and RAGAS 0.2 drives the automated evaluation. Analyst access runs through a FastAPI backend with a Streamlit workbench; the agent interface uses the MCP Python SDK. Every component of the evaluated pipeline is open-source and runs locally (C6); a hosted model appears only in the optional extended mode (Section 3.7.1) and in the ablation configuration V5, both outside the evaluated path. Table 6 maps the modules to their functions.

Path	Function	Details
src/cti_rag/ingestion/	source acquisition	downloaders and parsers for NVD, CISA KEV, CISA advisories, and MISP; unified document model
src/cti_rag/retrieval/	indexing and retrieval	CTI-aware BM25 indexer, ChromaDB collection, hybrid retriever with RRF and reranking
src/cti_rag/rag/	orchestration and generation	query router, triage templates (L1/L2), prompts, citation normalization, abstention
src/cti_rag/evaluation/	evaluation harness	RAGAS runner, field coverage, analyst rubric
src/cti_rag/api/, ui/	analyst access	FastAPI backend, Streamlit workbench
src/cti_rag/mcp/	agent interface	stdio MCP server with a deterministic query tool

Table 6: Prototype components with path, function, and technical details

The prototype was developed and operated on consumer hardware: a MacBook Air with an Apple M4 chip and 24 GB of unified memory, running macOS 26. All models execute locally — the generation model through the Ollama runtime, the embedding and reranking models through

`sentence-transformers` on Apple's Metal backend. No dedicated GPU server is involved: the complete pipeline, including 8B-model generation, runs on analyst-grade hardware, which substantiates the local-deployment criterion (C6). The exact software versions of the evaluation runs are listed in Section 4.1.

Table 7 documents the component selection. The selection is justified against the design criteria rather than a formal scoring model; for the generation model, the local-versus-hosted decision is additionally checked empirically in the ablation (Section 4.3). The embedding model was chosen for its retrieval quality relative to its footprint on the MTEB benchmark (Muennighoff et al. 2022).

Component	**Selected**	**Alternatives considered**	**Rationale**
Generation model	Llama 3.1 8B Instruct (Ollama, q5_K_M)	Mistral 7B, Qwen 2.5 7B; hosted model (V5)	C6: fits 24 GB unified memory; empirical check in Sec. 4.3
Embedding model	BGE-small-en-v1.5 (384 dim.)	e5-small-v2, all-MiniLM-L6-v2	C2, C6: MTEB retrieval quality at small footprint
Sparse index	rank-bm25 with custom CTI tokenizer	Elasticsearch, OpenSearch	C2, C6: exact identifier matching, no server dependency
Vector store	ChromaDB	FAISS, Qdrant	C6: embedded, persistent, open-source
Reranker	ms-marco-MiniLM-L-6-v2	bge-reranker-base	C2: precision gain at acceptable latency
Orchestration	LangChain 0.3	LlamaIndex, Haystack	C6: mature Ollama, Chroma, and Hugging Face integrations

Table 7: Technology selection with rationale against the design criteria

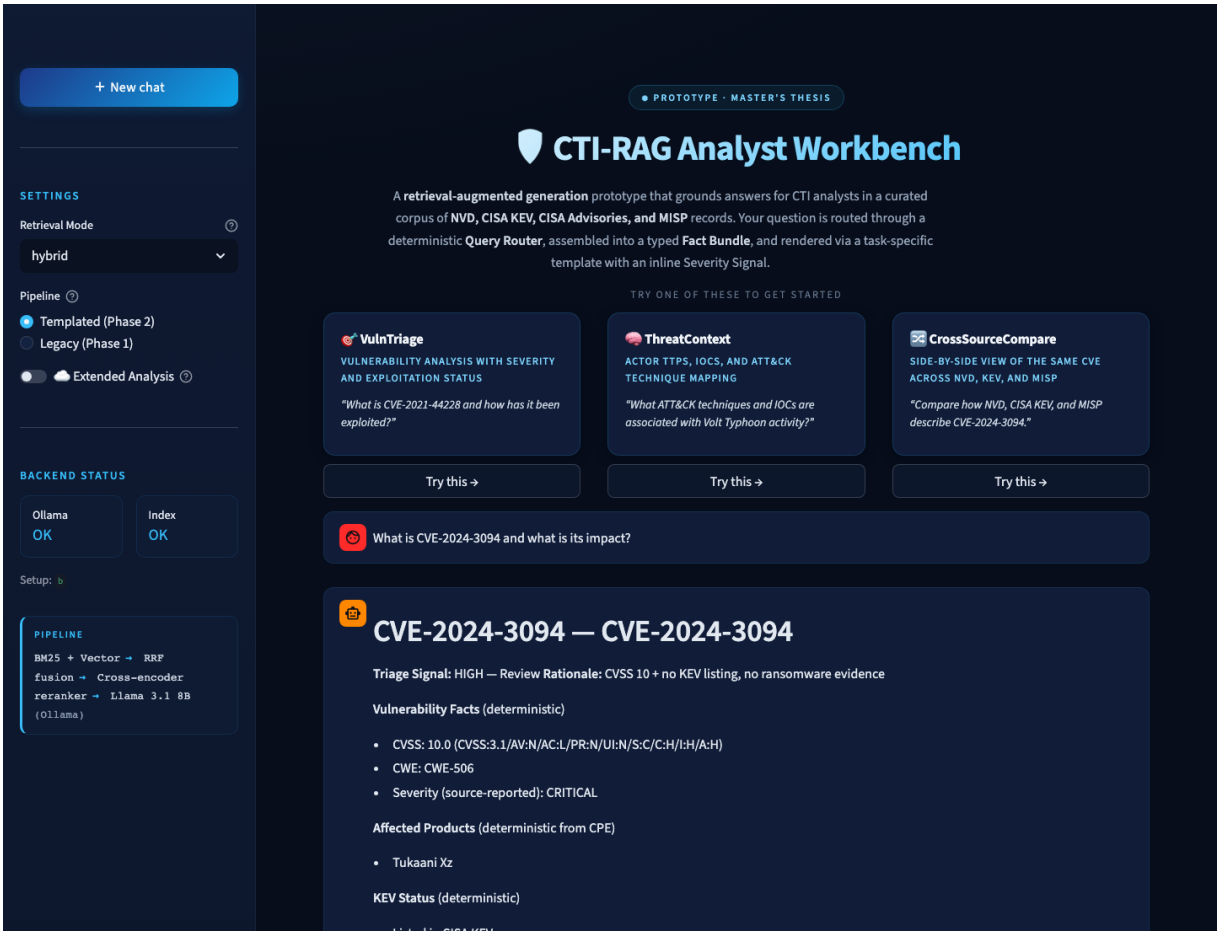

Figure 5: Analyst workbench of the prototype: task-template cards, pipeline and backend status in the sidebar, and the head of a grounded response

3.7.1 Agent interface via MCP

Beyond the analyst-facing workbench, the prototype exposes its pipeline to agentic clients through a Model Context Protocol (MCP) stdio server. The server publishes a single deterministic tool, `cti_query_tool`, which runs the query router, hybrid retrieval, fact-bundle extraction, and Layer-1 rendering — but no LLM generation — and responds in roughly 1.2 seconds. This cut is deliberate: full generation on the local model takes a median of 85.2 seconds and up to three minutes per answer, which exceeds typical MCP client timeouts, whereas the deterministic fact layer is fast, reproducible, and grounded by construction. An extended assist mode that adds hosted generation on top of this interface exists as a documented feature but is not part of the evaluation; it is discussed as future work (Section 5.3).

3.8 Evaluation design

The evaluation design fixes four elements before any result is interpreted: the query set, the metrics, the configurations under comparison, and the protocol.

3.8.1 Query set construction

The query set contains 28 queries across the four CTI task types: vulnerability analysis (6), IOC enrichment (7), TTP correlation (7), and cross-source comparison (8). Table 8 gives one example per type; Appendix C lists the full set. Each query defines its expected router template, a ground truth as a list of individual facts, the fields a complete answer must cover, and a difficulty level. The router assignment is asserted by unit tests, so every query reaches the template it was designed for.

Every ground-truth fact is verifiable against the indexed snapshot: the ground truths were reviewed against the corpus, open enumeration questions were narrowed to named entity sets so that the correct answer set is uniquely retrievable, and facts without snapshot backing were removed. Temporal references use absolute dates aligned with the snapshot cutoff of 2025-12-31.

Task type	**Queries**	**Example query**
Vulnerability analysis	6	“What is CVE-2024-3094 and what is its impact?”
IOC enrichment	7	. . .
TTP correlation	7	. . .
Cross-source comparison	8	. . .

Table 8: Evaluation query set by CTI task type (28 queries in total)

3.8.2 Evaluation metrics

Three metric groups cover complementary quality aspects (Table 9). The RAGAS framework (Es et al. 2023) scores faithfulness, answer relevancy, answer correctness, context precision, and context recall; an LLM judge performs the scoring, and the ground-truth facts of the query set serve as references where the metric requires them. The judge model (gpt-4o-mini) deliberately differs from the hosted generation model of the ablation (V5) to avoid self-preference bias. RAGAS receives the Layer-2 narrative without the gaps section and without template-mandated absence statements: the judge’s non-committal classifier nulls answers that contain absence phrasing, and Layer 1 is metadata-grounded by construction, so it needs no LLM-based grounding check.

Field coverage measures task-specific completeness: for each query, it checks deterministically which of the expected fields, such as CVSS score, affected products, or KEV remediation actions, appear in the answer. This metric operationalizes criterion C4 beyond generic quality scores. The analyst rubric assesses each answer on four dimensions with a 0–3 scale: prioritization, actionability, completeness, and traceability.

Metric	**Measures**	**Basis**
Faithfulness	claims supported by the retrieved context	judge
Answer relevancy	answer addresses the query	judge
Answer correctness	agreement with the ground-truth facts	judge + GT
Context precision	retrieved passages relevant to the query	judge
Context recall	ground-truth facts covered by the context	judge + GT
Field coverage	expected task fields present in the answer	deterministic
Analyst rubric	prioritization, actionability, completeness, traceability (0–3 each)	rubric

Table 9: Evaluation metrics of the final series

3.8.3 Baseline and variant configurations

Table 10 defines the six configurations. They vary along two axes: the retrieval strategy (none, sparse, dense, hybrid) and the generation mode (legacy prompt or templated pipeline, Section 3.6). V0 instantiates the non-retrieval baseline a_0 of the task formulation. V1–V3 isolate the retrieval strategies under the legacy prompt. V4 adds the templated pipeline on top of the hybrid retriever and is the primary configuration of this thesis; the comparison V3 versus V4 therefore isolates the effect of the templated generation. V5 repeats V4 with a hosted generation model as an ablation. Cross-encoder reranking is uniformly active in V1–V5 and deliberately not ablated separately. A second index setup (Setup A) was used during development and is descoped from the final evaluation series.

ID	Configuration	Retrieval	Generation	n
V0	no-retrieval baseline	—	local 8B, legacy	3
V1	BM25 only	sparse	local 8B, legacy	3
V2	vector only	dense	local 8B, legacy	3
V3	hybrid (RRF)	sparse + dense	local 8B, legacy	3
V4	hybrid + templated pipeline	sparse + dense	local 8B, templated	3
V5	V4 with hosted generator	sparse + dense	Haiku 4.5, templated	3

Table 10: Configuration matrix of the final evaluation series. Cross-encoder reranking is active in V1–V5 and not ablated separately.

3.8.4 Experimental protocol

All configurations run on the same frozen state: the index of Section 3.4, the prompts of Appendix B, and the retrieval parameters of Table 5 are tagged and unchanged across the series. Every configuration, including the hosted ablation, is executed three times, because single runs scatter by about ± 0.05 through generation variance; all reported numbers are three-run means with standard deviations. Generation uses a temperature of 0.1; the judge model and its configuration stay fixed across all runs.

The scenario-based assessment in Section 4.4 is a critical self-evaluation: the author rates representative outputs along the predefined rubric of Appendix C. This design keeps the judgment criteria fixed before the results and is named as a limitation in Section 5.2. All evaluation artifacts, including raw scores and run logs, are versioned in the repository (Appendix A) together with the batch runner that reproduces the series.

4 Results and critical appraisal

4.1 Experimental setup

The final evaluation series ran on the frozen `evaluated-v1` state: all six configurations of Table 10 were executed against the index of Section 3.4, on the machine described in Section 3.7, over the full query set with three runs per configuration (Section 3.8.4). A batch runner executes the series end-to-end; the raw scores and run logs are versioned in the repository (Appendix A).

Table 11 lists the models and software versions involved; the complete environment is pinned in `requirements-evaluated-v1.lock` in the repository.

Component	**Version / specification**
Generation model (local)	`llama3.1:8b-instruct-q5_K_M`, Ollama 0.32
Generation model (hosted, V5)	Claude Haiku 4.5 (via OpenRouter)
Embedding model	BAAI/bge-small-en-v1.5
Reranker	cross-encoder/ms-marco-MiniLM-L-6-v2
Evaluation judge	gpt-4o-mini (via OpenRouter)
Software	Python 3.12.13, LangChain 0.3.28, ChromaDB 0.6.3, RAGAS 0.2.15, sentence-transformers 3.4.1, torch 2.11.0

Table 11: Models and software versions of the final evaluation runs, consolidated as the reproducibility record of the series

Two remarks bound the comparability of the results. First, results produced before the re-index of 2026-07-20 are not comparable to the final series: a snapshot-filter defect had silently dropped 3,081 NVD documents from the index, and the final series runs on the repaired state described in Section 3.4. Second, the RAGAS input rule of Section 3.8.2 applies to all scores: rated is the Layer-2 narrative, not the deterministic Layer-1 block.

4.2 Generation quality

Table 12 reports the RAGAS scores of all configurations. Every value is a three-run mean with standard deviation (Section 3.8.4).

Pipeline	n	Faithfulness	Answer Relevancy	Answer Correctness	Context Precision	Context Recall
Templated hybrid (local 8B)	3	0.662 ± 0.020	$\mathbf{0.739 \pm 0.024}$	$\mathbf{0.518 \pm 0.016}$	0.733 ± 0.003	$\mathbf{0.610 \pm 0.009}$
Legacy hybrid (local 8B)	3	0.689 ± 0.023	0.570 ± 0.032	0.500 ± 0.012	$\mathbf{0.749 \pm 0.005}$	0.498 ± 0.009
BM25 only	3	$\mathbf{0.797 \pm 0.001}$	0.598 ± 0.020	0.493 ± 0.011	0.735 ± 0.012	0.427 ± 0.012
Vector only	3	0.664 ± 0.005	0.419 ± 0.021	0.403 ± 0.008	0.593 ± 0.001	0.373 ± 0.007
No-retrieval baseline	3	—	0.685 ± 0.019	0.502 ± 0.014	—	—
Templated hybrid (hosted, Haiku 4.5)	3	0.821 ± 0.017	0.594 ± 0.048	0.561 ± 0.021	0.748 ± 0.021	0.597 ± 0.026

Table 12: RAGAS results per pipeline (28 queries, judge gpt-4o-mini). Rows correspond, top to bottom, to configurations V4, V3, V1, V2, V0, and V5 of Table 10. The BM25-only faithfulness (0.797) is an abstention artifact (4–5 abstentions per run with conservative, extractive answers), not a grounding advantage.

The templated pipeline (V4) reaches the best local values in answer relevancy (0.739), answer

correctness (0.518), and context recall (0.610), and it is the only local retrieval configuration that never abstains; the legacy prompt abstains on 7 of 28 queries. Its faithfulness (0.662) stays close to the legacy configuration (0.689).

Two values require care when reading the table. The highest faithfulness belongs to the BM25-only configuration (0.797); as noted in the caption, this reflects its abstention behavior, not better grounding: the configuration declines the queries it cannot support and answers the remaining ones close to the retrieved text. The relevancy of the hosted ablation (0.594) carries a measurement artifact[1] and is discussed with the ablation in Section 4.3.

The non-retrieval baseline (V0) reads well in isolation: its answer relevancy (0.685) and answer correctness (0.502) sit at the level of the grounded configurations. Faithfulness and the context metrics are undefined without retrieval; what the fluent surface hides becomes visible in the rubric results of Section 4.4. The vector-only configuration (V2) is the weakest across all metrics; Section 4.3 traces this to its failure on exact CTI identifiers.

4.3 Baseline comparison and ablation

The configuration matrix isolates three effects: retrieval versus parametric knowledge (V0 against the grounded configurations), the retrieval strategy (V1–V3), and the generation mode (V3 against V4). The hosted ablation (V5) adds the deployment dimension. All numbers refer to Tables 12 and 13.

Retrieval versus parametric knowledge. On this query set, retrieval buys verifiability rather than raw correctness. The baseline answers prominent, well-documented CVEs from parametric knowledge at a correctness level comparable to the grounded configurations (0.502 versus 0.518 for V4); the query set names prominent CVEs and therefore favors the baseline systematically, which Section 5.2 records as a limitation. The rubric separates the configurations where RAGAS does not: the baseline's traceability of 1.23 against 2.56 for V4 means that an analyst cannot verify the baseline's claims — and unverifiable output cannot be acted on in CTI work (B3, C1).

Retrieval strategy. Each single-strategy retriever fails in a characteristic way. The vector-only configuration collapses on exact CTI identifiers: it abstains on 14–15 of 28 queries per run and scores lowest on every metric, because embedding similarity does not reliably surface passages for identifier queries such as specific CVE numbers. The BM25-only configuration

[1] The relevancy filter that nulls non-committal answers is calibrated on the phrasing of the local 8B model. The hosted model formulates gap statements differently and is nulled on 5–8 of 28 queries per run. The filter stayed frozen for consistency across configurations.

holds identifier queries but loses semantic coverage: its context recall of 0.427 undercuts both hybrid configurations. The hybrid configuration removes the identifier failure and lifts context recall to 0.498, supporting the hybrid criterion (C2).

Generation mode. V3 and V4 share the same retriever, so their differences follow from the generation mode. The templated pipeline raises answer relevancy by 0.169 and context recall by 0.112, lifts the rubric total from 1.67 to 2.29, and reduces abstentions from seven to zero; the higher context recall likely reflects this abstention difference, since abstained queries leave ground-truth facts uncovered. This comparison carries the main design contribution of the thesis: the gains come from structuring the answer, not from changing the model or the retrieval.

Local versus hosted generation. Replacing the local 8B model with the hosted model improves faithfulness by 0.159, answer correctness by 0.043, and the rubric total from 2.29 to 2.39, with near-perfect traceability (2.93). It also reduces median generation latency from 85.2 to 6.4 seconds per query, a factor of thirteen. Answer relevancy is the one metric where the hosted model scores lower; the footnote in Section 4.2 traces this to the frozen relevancy filter rather than to answer quality. The price is the deployment model: hosted generation moves analyst queries off premises and gives up the local-deployment property that C6 demands. The evaluated prototype therefore keeps the local model as its primary configuration and treats hosted generation as an option whose benefit is now quantified.

4.4 Scenario-based evaluation of representative outputs

This section moves beyond offline metrics (C4). Three instruments assess the outputs from an analyst perspective: the four-dimension rubric of Section 3.8.2, the field-coverage measurement, and a walk-through of one representative output per task type. Full transcripts are reproduced in Appendix E, per-query details in Appendix D.

Pipeline	n	Prioritization	Actionability	Completeness	Traceability	Total
Templated hybrid	3	2.55 ± 0.02	1.77 ± 0.15	2.27 ± 0.05	2.56 ± 0.08	2.29 ± 0.06
Legacy hybrid	3	1.74 ± 0.02	1.10 ± 0.09	2.01 ± 0.08	1.82 ± 0.09	1.67 ± 0.06
No-retrieval baseline	3	1.54 ± 0.04	1.87 ± 0.07	2.02 ± 0.07	1.23 ± 0.02	1.66 ± 0.01
Templated hybrid (hosted, Haiku 4.5)	3	2.67 ± 0.04	1.90 ± 0.02	2.06 ± 0.02	2.93 ± 0.00	2.39 ± 0.01

Table 13: Analyst-oriented rubric scores (0–3 per dimension, three-run means). Rows correspond, top to bottom, to configurations V4, V3, V0, and V5 of Table 10; the single-strategy configurations V1 and V2 were not rubric-scored.

The templated pipeline reaches a rubric total of 2.29, compared to 1.67 for the legacy prompt and 1.66 for the non-retrieval baseline. The dimension profile of the baseline is the central finding of this table: its actionability (1.87) exceeds the legacy configuration (1.10) and almost matches the templated pipeline (1.77), while its traceability collapses to 1.23. The baseline produces recommendations that sound actionable but cannot be traced to evidence, precisely the failure mode that bottleneck B3 describes and criterion C1 is designed to prevent. The hosted ablation leads every dimension (total 2.39, traceability 2.93); Section 4.3 weighs this against its deployment cost.

Field coverage complements the rubric on the completeness side: across the final runs, the templated pipeline covers 87–89 % of the task-specific fields expected by the query set.

The following subsections examine one representative output per task type, each assessed along the rubric dimensions.

4.4.1 Vulnerability analysis

Query `vuln_001` asks what CVE-2024-3094, the xz-utils supply-chain backdoor, is and what its impact is. Figure 6 shows the workbench response to a live query against the evaluated configuration; the status pills confirm the templated pipeline and the local model.

Three properties of this response carry the assessment. First, the deterministic head grades the finding as HIGH and states its rationale (CVSS 10.0, but no KEV listing and no ransomware evidence in the corpus snapshot), so the prioritization is explained rather than merely asserted. Second, every narrative claim ends in a citation, and the source panel distinguishes the two cited passages from the three retrieved but uncited ones; the analyst sees both what the answer uses and what it ignored (transparency, C1, C5). Third, the gaps section reports the absence of indicators and CWE-specific mitigations in the retrieved context instead of filling these slots from parametric knowledge.

4.4.2 IOC enrichment

Query `ioc_003` asks what is known about the MOVEit Transfer vulnerability CVE-2023-34362 and its exploitation; the router assigns the vulnerability-triage template. The excerpt shows the deterministic head and two narrative claims; Appendix E reproduces the full answer.

ioc_003 — “What is known about the MOVEit Transfer vulnerability CVE-2023-34362 and its exploitation?” (V4, excerpt)

Triage Signal: CRITICAL — Immediate attention
Rationale: CVSS 9.8 + active KEV listing + known ransomware use
Vulnerability Facts: CVSS 9.8; CWE-89; severity CRITICAL

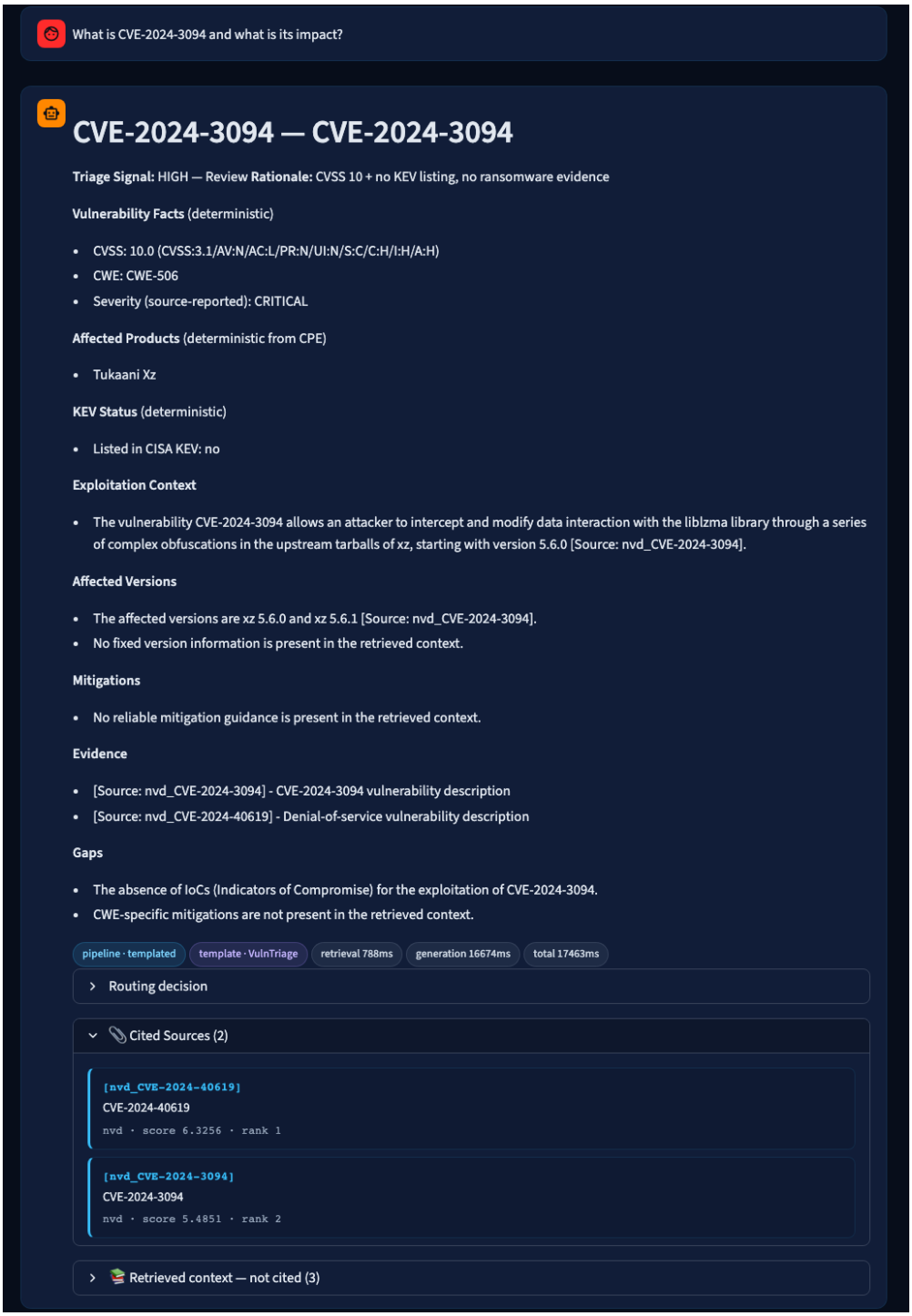

Figure 6: Workbench response to query `vuln_001` (live query, configuration V4). The deterministic head reports the triage signal with its rationale; every narrative claim carries a citation; the source panel separates cited from retrieved but uncited context.

KEV Status: listed 2023-06-02; known ransomware use: yes; CISA required action: Apply updates per vendor instructions.

Exploitation Context

- The vulnerability CVE-2023-34362 allows an unauthenticated attacker to gain unauthorized access to MOVEit Transfer's database. `[Source: nvd_CVE-2023-34362]`
- There is evidence of active exploitation in May and June 2023 via HTTP or HTTPS. `[Source: nvd_CVE-2023-34362]`

Gaps: Indicators of Compromise (IoCs) for this specific vulnerability [. . .]

The answer passes seven of eight field-coverage checks. The deterministic head answers the triage question at a glance (prioritization), and the only recommended action is the one the KEV record supports (actionability without invention). The gaps section names what the corpus does not provide (concrete indicator values for this CVE) instead of fabricating them; for an enrichment query, this explicit statement of absence is the intended behavior of the grounding design (transparency, C1).

4.4.3 TTP correlation

Query `ttp_005` asks which MITRE ATT&CK techniques relate to ransomware impact; the router assigns the threat-context template.

ttp_005 — "Which MITRE ATT&CK techniques relate to ransomware impact, specifically data encryption for impact?" (V4, excerpt)

ATT&CK Context: Tactic: Impact (TA0040)

Threat Actors and Malware

- Hive actors: attributed to the retrieved context `[Source: cisa_advisory_AA22-321A_mitre_att_ck_techniques]`
- Play ransomware group: attributed to the retrieved context `[Source: cisa_advisory_AA23-352A_impact]`
- Rhysida ransomware actors: attributed to the retrieved context `[Source: cisa_advisory_AA23-319A_technical_details]`

Defensive Guidance

- Monitor for exploitation of CVE-2021-34473, CVE-2021-34523, CVE-2021-31207, and CVE-2021-42321 to prevent Hive actors' initial access `[Source: cisa_advisory_AA22-321A_mitre_att_ck_techniques]`

The correlation value lies in the aggregation: the answer consolidates five CISA advisories into one view of actors, techniques, and defensive guidance for the Impact tactic, with each attribution citing the advisory section it comes from. This task family is also where the generation

modes separate most clearly: the legacy prompt is unstable on threat-context queries, while the templated pipeline stays consistent.

4.4.4 Cross-source comparison

Query `cross_003` asks for the NVD severity ratings and the KEV exploitation status of three 2023 CVEs; the router assigns the cross-source-comparison template.

cross_003 — "For CVE-2023-22515, CVE-2023-34362, and CVE-2023-3519, which severity ratings does NVD report and what exploitation status does the CISA KEV catalog record?" (V4, excerpt)

Summary: Entities evaluated: 3; with data from NVD and CISA KEV: 3

Comparison Table (deterministic)

CVE	CVSS	KEV listed	Ransomware	Vendor
CVE-2023-22515	9.8	yes	yes	Atlassian
CVE-2023-34362	9.8	yes	yes	Progress
CVE-2023-3519	9.8	yes	yes	Citrix

Source Agreement

- The sources agree that the severity of the vulnerabilities is CRITICAL.
- NVD records CVSS 9.8 for CVE-2023-22515 `[Source: nvd_CVE-2023-22515]`; CISA KEV does not provide a CVSS score.

The deterministic comparison table answers the question directly, and the agreement section makes source differences explicit instead of averaging them away. The same answer also shows the residual weakness of the narrative layer: its gaps section claims that the exploitation status of CVE-2023-34362 is missing, although the Layer-1 table records the KEV listing of exactly that CVE. The layered design contains such inconsistencies (the deterministic facts remain correct) but does not eliminate them; Section 5.2 returns to this point.

4.5 Assessment against the design criteria

Table 14 assesses the prototype against the six criteria of Section 3.2, with the evidence for each judgment referenced instead of repeated.

Criterion	Status	Evidence	Addresses
C1 Evidence-linked outputs	fulfilled	Tab. 13; Fig. 6	B3
C2 Hybrid retrieval	fulfilled	Sec. 4.3	B1
C3 Source integrity	partially	Sec. 3.4; Sec. 5.2	G3
C4 Scenario-based evaluation	fulfilled	Sec. 4.4	G2
C5 Analyst-oriented presentation	fulfilled	Tab. 13	B1
C6 Open-source, local deployment	fulfilled	Sec. 3.7	Scope

Table 14: Assessment of the prototype against the design criteria

The fulfilled criteria close the loop to the workflow bottlenecks of Section 2.1. The verification bottleneck (B3) is where the prototype changes analyst work most clearly: enforced citations raise traceability from 1.23 (baseline) to 2.56, and the interface separates cited from uncited context, so verification becomes an inspection step instead of a research task. The overload bottleneck (B1) is addressed from two sides: hybrid retrieval keeps identifier queries reliable where single strategies fail, and the templated output presents each answer in a fixed triage structure. The recency bottleneck (B2) is addressed by design, since the offline path can re-index updated sources at any time, but not demonstrated empirically: the evaluation deliberately ran on a frozen snapshot for reproducibility.

Two judgments require qualification. C3 is fulfilled only partially: ingestion is restricted to curated sources and every passage keeps its provenance, but the prototype does not assess source credibility at query time, as stated in Section 3.2. For C5, the rubric confirms the presentation gains overall (2.29 versus 1.67), yet actionability is the weakest templated dimension (1.77): the mitigation sections only relay what the corpus offers, and for many CVE entries that is a single KEV remediation line or an explicit statement of absence. Both boundaries carry over to the limitations in Section 5.2.

One scope note guards the C6 judgment: it refers to the evaluated configurations V0–V4, which contain no hosted dependency. The hosted model of the ablation (V5) is an evaluation instrument — the measured counterfactual of giving up local deployment — and the evaluation judge is measurement instrumentation, not part of the artifact. The optional extended mode calls a hosted model and is documented as a feature outside the evaluated configuration (Section 3.7.1).

4.6 Reference model for CTI-RAG integration

The empirical findings generalize beyond the prototype into a reference model for integrating RAG into CTI and threat-hunting workflows (Figure 7). The model organizes an integration

into four layers inside a local deployment boundary: curated sources, a provenance-preserving knowledge layer, a layered reasoning stage, and an analyst interaction layer, flanked by an evaluation and governance function. Eight design principles, marked P1–P8 in the figure, locate the empirically grounded decisions; Table 15 states each principle with the evidence it rests on.

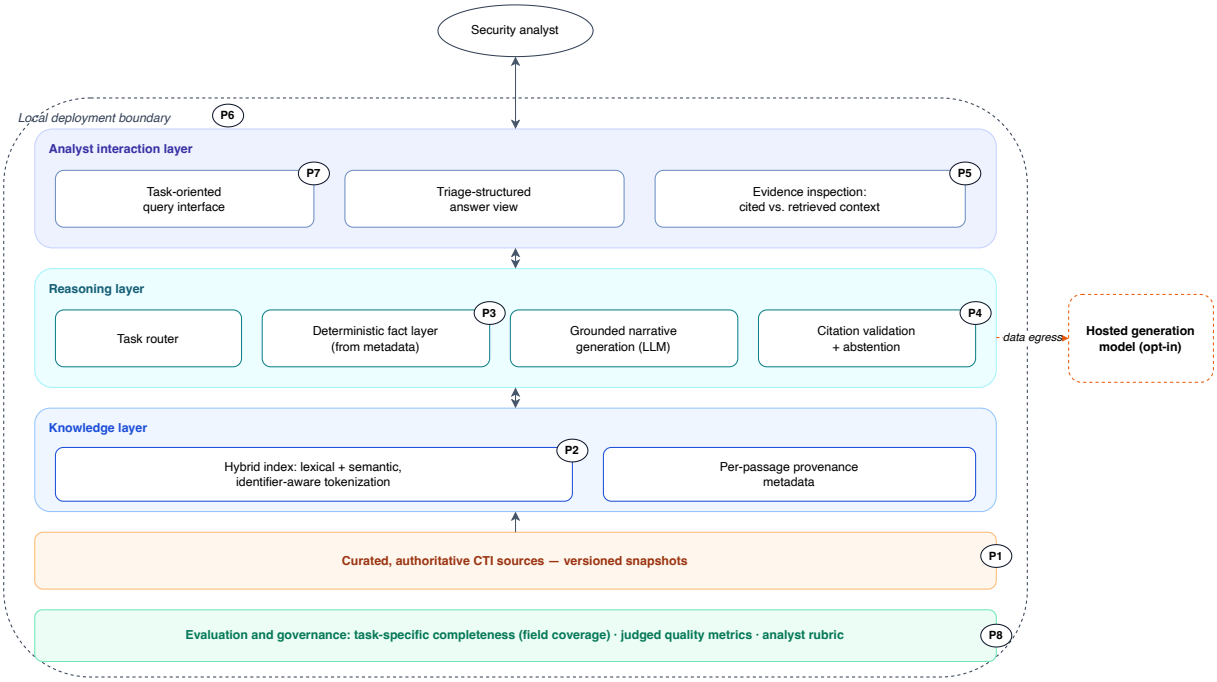

Figure 7: Reference model for integrating RAG into CTI workflows. Badges P1–P8 locate the design principles of Table 15.

ID	Design principle	Empirical basis
P1	Restrict ingestion to curated, authoritative sources and version the corpus as snapshots	G3 (Sec. 2.3); C3 assessment (Sec. 4.5)
P2	Combine lexical and semantic retrieval and preserve CTI identifiers as single tokens	single-strategy failures (Sec. 4.3)
P3	Render verifiable facts deterministically from metadata; let the LLM only narrate	V3 vs. V4 (Sec. 4.3)
P4	Enforce citations at generation time, validate them afterwards, and abstain without evidence	traceability 2.56 vs. 1.23 (Tab. 13)
P5	Expose evidence for inspection and separate cited from retrieved but uncited context	scenario walk-throughs (Sec. 4.4)
P6	Deploy locally by default; treat hosted generation as an explicit, quantified opt-in	V4 vs. V5 trade-off (Sec. 4.3)
P7	Structure the interaction around recurring analyst task types	per-template consistency (Sec. 4.4.3)
P8	Build task-specific evaluation into the integration; generic metrics miss operational utility	RAGAS vs. rubric divergence of the baseline (Sec. 4.2, Tab. 13)

Table 15: Design principles of the reference model with their empirical basis

The model reads as adoption guidance: an organization integrating RAG into CTI work starts at the source layer and moves upward, and every principle names the failure it prevents: P2 the identifier collapse of semantic-only retrieval, P3 and P4 the unverifiable fluency of ungrounded generation, P6 the silent loss of data control. The model is derived from one prototype, one corpus, and one query set; transferring it to other corpora and organizations requires validation, which Section 5.2 records and Section 5.3 takes up.

4.7 Answering the research questions

The boxes below answer the research questions of Section 1.3; each answer names the evidence it rests on.

MRQ

How can a Retrieval-Augmented Generation (RAG) system leveraging Large Language Models improve the quality, reliability, and operational usefulness of Cyber Threat Intelligence (CTI) analysis and Threat-Hunting workflows?

Answer: By grounding a locally deployed LLM in a curated, provenance-preserving corpus

through hybrid retrieval and a layered, citation-enforcing generation pipeline. The improvement lies less in raw answer correctness than in verifiability and task fit: grounded answers can be checked claim by claim, abstain without evidence, and arrive in a triage-ready structure. The reference model consolidates these properties into transferable design principles.

Evidence:

- Correctness at baseline level, traceability transformed (1.23 to 2.56) — Sections 4.2 and 4.3, Table 13
- Five of six design criteria fulfilled, bottlenecks B1 and B3 addressed — Section 4.5
- Reference model with eight principles — Section 4.6

SRQ1 — Information retrieval and answer quality

To what extent does retrieval augmentation improve the completeness, factual accuracy, and recency of CTI information generated by an LLM compared to a non-retrieval baseline?

Answer: Retrieval augmentation improves completeness and relevance, while factual accuracy on this query set stays comparable to the parametric baseline, which the prominent CVEs of the query set favor. The decisive gain is that retrieved facts are verifiable and current by construction: the indexed snapshot, not the generation model's training cutoff, determines what the prototype knows, and claims without evidence are abstained from rather than generated.

Evidence:

- Answer relevancy 0.739 vs. 0.685, correctness 0.518 vs. 0.502, context recall 0.610 — Table 12
- Field coverage 87–89 % of expected task fields — Section 4.4
- Traceability 2.56 vs. 1.23; zero abstentions with explicit absence statements — Table 13, Section 4.2

SRQ2 — Practical utility and workflow support

Which types of CTI analysis tasks can be effectively supported by a RAG-based LLM system, and how well do the generated outputs align with the informational needs of threat-hunting workflows?

Answer: All four task types of the query set are supported end to end. Support is strongest where deterministic structure carries the answer — vulnerability analysis and cross-source comparison — and remains consistent on TTP correlation, where the unstructured prompt becomes unstable. The main boundary is actionability: mitigation guidance can only relay what the corpus documents.

Evidence:

- Vulnerability analysis: triage signal with rationale, cited evidence — Section 4.4.1
- IOC enrichment: seven of eight expected fields, absence stated instead of fabricated

— Section 4.4.2
- TTP correlation: five advisories consolidated; templated pipeline consistent where legacy is unstable — Section 4.4.3
- Cross-source comparison: deterministic comparison table, source disagreements made explicit — Section 4.4.4
- Rubric total 2.29 vs. 1.67 (legacy) and 1.66 (baseline); actionability weakest at 1.77 — Table 13

SRQ3 — Conceptual design

How can empirical insights from the RAG-LLM prototype be translated into actionable design principles for integrating generative AI into CTI and threat-hunting workflows?

Answer: The evaluation results condense into eight design principles — from curated, provenance-preserving ingestion through identifier-aware hybrid retrieval and layered, citation-enforcing generation to local-first deployment with hosted generation as a quantified opt-in — arranged in a four-layer reference model with an evaluation and governance function. Each principle names the empirically observed failure it prevents.

Evidence:
- Principles P1–P8 with per-principle empirical basis — Table 15
- Reference model locating the principles — Figure 7

5 Conclusion, limitations and outlook

5.1 Summary and contributions

...

5.2 Limitations

...

5.3 Future work

...

Bibliography

Agnihotram, Gopichand and Joydeep Sarkar (2025). *Evaluating Precision and Recall at Retrieval Time in Retrieval-Augmented Generation (RAG) Systems*.

Akkiraju, Rama et al. (2024). *FACTS About Building Retrieval Augmented Generation-based Chatbots*. arXiv preprint arXiv:2407.07858.

Alhuzali, Abeer (2025). *LLM-Powered Threat Intelligence: A Retrieval-Augmented Generation Approach for Cyber Attack Investigation*.

Brown, Simon (2018). *The C4 Model for Visualising Software Architecture*. https://c4model.com.

Chen, Jiawei et al. (2023). *Benchmarking Large Language Models in Retrieval-Augmented Generation*. arXiv preprint arXiv:2309.01431.

Divakaran, Dinil Mon and Sai Teja Peddinti (2024). *LLMs for Cyber Security: New Opportunities*. arXiv preprint arXiv:2404.11338.

Es, Shahul et al. (2023). *RAGAS: Automated Evaluation of Retrieval Augmented Generation*. arXiv preprint arXiv:2309.15217.

Fayyazi, Reza, Rozhina Taghdimi, and Shanchieh Jay Yang (2024). *Advancing TTP Analysis: Harnessing the Power of Large Language Models with Retrieval Augmented Generation*. arXiv preprint arXiv:2401.00280.

Gao, Yunfan et al. (2024). *Retrieval-Augmented Generation for Large Language Models: A Survey*. arXiv preprint arXiv:2312.10997.

Ismail, Muhammad et al. (2025). *Enhancing Security Operations Center: Wazuh Security Event Response with Retrieval-Augmented-Generation-Driven Copilot*.

Lekssays, Ahmed et al. (2025). *TechniqueRAG: Retrieval Augmented Generation for Adversarial Technique Annotation in Cyber Threat Intelligence Text*. arXiv preprint arXiv:2505.11988.

Lewis, Patrick et al. (2020). “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”. In: *Advances in Neural Information Processing Systems*. Vol. 33, pp. 9459–9474.

Liu, Yang et al. (2023). *Trustworthy LLMs: A Survey and Guideline for Evaluating Large Language Models’ Alignment*. arXiv preprint arXiv:2308.05374.

Muennighoff, Niklas et al. (2022). *MTEB: Massive Text Embedding Benchmark*. arXiv preprint arXiv:2210.07316.

Palacín, Valentina (2021). *Practical Threat Intelligence and Data-Driven Threat Hunting*. Packt Publishing.

Paul, Shuva et al. (2025). *LLM-Assisted Proactive Threat Intelligence for Automated Reasoning*.

Rajapaksha, Sampath, Ruby Rani, and Erisa Karafili (2024). *A RAG-Based Question-Answering Solution for Cyber-Attack Investigation and Attribution*. arXiv preprint arXiv:2408.06272.

Rastogi, Nidhi and Md Tanvirul Alam (2023). *Cyber Threat Intelligence for SOC Analysts*.

Saad-Falcon, Jon et al. (2024). "ARES: An Automated Evaluation Framework for Retrieval-Augmented Generation Systems". In: *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*.

Sun, Nan et al. (2023). "Cyber Threat Intelligence Mining for Proactive Cybersecurity Defense: A Survey and New Perspectives". In: *IEEE Communications Surveys & Tutorials* 25.3, pp. 1748–1774.

Tseng, PeiYu et al. (2024). *Using LLMs to Automate Threat Intelligence Analysis Workflows in Security Operation Centers*.

Xu, Hanxiang et al. (2025). *Large Language Models for Cyber Security: A Systematic Literature Review*. arXiv preprint arXiv:2405.04760.

Yu, Hao et al. (2024). *Evaluation of Retrieval-Augmented Generation: A Survey*. arXiv preprint arXiv:2405.07437.

List of figures

List of tables

Documentation table of AI-based tools

AI-Based Tool	Intended Use	Prompt, Source, Page, Paragraph. . .
DeepL Translate	Translation of an article in English	Source (XXX), Chapter X on page X-X
Chat GPT 4.0	Grammar and Spelling	"Please list issues with spelling and grammar in the following text: ...", Entire document

List of abbreviations

AI	Artificial Intelligence
ARES	Automated RAG Evaluation System
ATT&CK	Adversarial Tactics, Techniques, and Common Knowledge
BM25	Best Match 25
CISA	Cybersecurity and Infrastructure Security Agency
CTI	Cyber Threat Intelligence
CVE	Common Vulnerabilities and Exposures
IOC	Indicator of Compromise
KEV	Known Exploited Vulnerabilities
LLM	Large Language Model
MCP	Model Context Protocol
MISP	Malware Information Sharing Platform
MRQ	Main Research Question
MRR	Mean Reciprocal Rank
nDCG	Normalized Discounted Cumulative Gain
NVD	National Vulnerability Database
RAG	Retrieval-Augmented Generation
RAGAS	Retrieval-Augmented Generation Assessment
SOC	Security Operations Center
SRQ	Sub-Research Question
TTP	Tactics, Techniques, and Procedures

A Prototype repository

The source code of the prototype is available in the following GitHub repository:

https://github.com/bhxinderj/cti-rag-analyst-support

The repository contains the implementation of the prototype described in this thesis, including configuration files, setup instructions, and supplementary materials for reproducing the main system components. In particular, it includes artifacts related to data ingestion and preprocessing, retrieval and ranking, answer generation, and selected evaluation components.

B Prompt templates

This appendix reproduces the prompt templates of the prototype verbatim from `src/cti_rag/rag/prompts.py` and `src/cti_rag/rag/templates.py` at the evaluated state (`evaluated-v1`). Listing B.1 is the system prompt of the legacy mode (V1–V3), Listing B.2 the corresponding user-message template, Listing B.3 the system prompt of the templated mode (V4/V5), and Listing B.4 the prompt of the non-retrieval baseline (V0). The per-template user messages of the templated mode, including the pre-rendered Layer-1 blocks and target section lists, are constructed programmatically; see the repository (Appendix A).

```
You are a Cyber Threat Intelligence (CTI) analyst assistant. Your role
is to help security analysts by providing accurate, source-grounded
intelligence based only on the retrieved context.

Rules:
1. Use ONLY the retrieved context. Do not use prior knowledge or fill
   gaps from memory.
2. If the context is insufficient, say so explicitly and name what is
   missing.
3. Every factual statement, recommendation, or interpretation that
   comes from the context MUST end with an inline citation using the
   exact Citation Label shown for that chunk. Use the short bracket
   form: [<citation_label>]. Example:
   "Apply updates per vendor instructions [cisa_kev_CVE-2024-1709]."
   "CVE-2024-3094 is a supply-chain backdoor in xz-utils
   [nvd_CVE-2024-3094]."
4. Never invent a citation label. Only use Citation Labels that appear
   verbatim in the retrieved context.
5. Every bullet and every sentence under Summary, Why it matters,
   Recommended actions / Mitigations, and Evidence MUST end with at
   least one [<citation_label>] bracket. Lines without a bracket will
   be dropped.
6. If multiple chunks support one claim, chain their citations:
   [<label_1>] [<label_2>].
7. Do not use Markdown bold headers like **Summary:**. Use plain
   section labels exactly:
   Summary:
   Why it matters:
   Recommended actions / Mitigations:
```

```
   Evidence:
   optional: Unknowns / Gaps:
8. Use the same structure for exact CVE questions and broader CTI
   analyst questions.
9. Put at most one grounded claim per bullet or line. Do not combine
   multiple factual claims in one sentence.
10. In Recommended actions / Mitigations, include only actions
    explicitly supported by the context. If none are supported, write
    exactly: No reliable mitigation guidance is present in the
    retrieved context.
11. In Evidence, list short evidence bullets grounded in the retrieved
    context with citations.
12. Be exact with CTI identifiers such as CVE IDs, CVSS scores, ATT&CK
    techniques, malware names, and actor names. These are NOT
    citations - always add a separate [<citation_label>] bracket after
    them.
```

Listing B.1: Legacy-mode system prompt (V1–V3), complete

```
Based on the retrieved context above, answer the following question:

{query}

Use this response format with plain labels (no Markdown bold):
Summary:
Why it matters:
Recommended actions / Mitigations:
Evidence:
optional: Unknowns / Gaps:

Citation rules - read carefully:
- End every claim with one or more [<citation_label>] brackets from
  the retrieved context.
- Use short bracket form like [nvd_CVE-2024-3094] - not [Source: ...]
  and not [1], [2].
- Remember: ATT&CK codes like [T1059] or CVE IDs in brackets are NOT
  citations. Always add an extra [<citation_label>] after them.
- Only use Citation Labels that appear verbatim in the retrieved
  context above.
- Lines without a [<citation_label>] bracket will be discarded from
  the final answer.
```

Listing B.2: Legacy-mode user-message template

```
You are a Cyber Threat Intelligence (CTI) analyst assistant operating
in Triage Card mode.
```

```
You will receive:
1. A pre-rendered Layer-1 block that already contains deterministic
   facts (CVE-ID, CVSS, KEV status, severity signal, comparison
   tables). Do NOT repeat, re-state, or modify this block.
2. A list of retrieved context chunks with explicit citation labels.
3. A list of target sections to produce.

Rules:
1. Produce ONLY the target sections listed in the user message, in
   order.
2. Use ONLY the retrieved context. Do not use prior knowledge.
3. Every claim line MUST end with an inline citation of the form
   [Source: <citation_label>], taken verbatim from the "Allowed
   Citation Labels" list in the user message. Uncited claim lines will
   be rejected downstream and replaced with an explicit grounding
   note, so it is always better to omit a claim than to assert it
   without a citation.
4. Keep to one grounded claim per bullet or line. If a bullet would
   make two independent claims, split it into two bullets, each with
   its own [Source: ...].
5. If you cannot find support in the retrieved context for a claim you
   would otherwise make, do NOT assert it. Either omit it entirely or
   move it to the "Unknowns / Gaps" section as a statement of absence.
6. If a whole section has no support in the retrieved context, write
   exactly: "No grounded information in the retrieved context."
7. Headers, bullet scaffolding, and explicit statements of absence in
   "Unknowns / Gaps" do not require citations.
8. Be exact with CTI identifiers (CVE IDs, CVSS scores, ATT&CK
   techniques, malware names, actor names) - copy them verbatim from
   the context.
```

Listing B.3: Templated-mode system prompt (V4/V5), complete

```
You are a Cyber Threat Intelligence (CTI) analyst assistant. Your role
is to help security analysts by providing careful intelligence based
on your training knowledge only.

Rules:
1. Answer using your training knowledge only.
2. Do not use citations.
3. Be explicit when details are uncertain, missing, or time-sensitive.
4. Keep the response in this structure:
   Summary:
   Why it matters:
   Recommended actions / Mitigations:
   Evidence:
```

```
   Unknowns / Gaps:
5. Evidence should describe the basis of your answer without source
   citations.
6. Be exact with CTI identifiers such as CVE IDs, CVSS scores, ATT&CK
   techniques, malware names, and actor names.
```

Listing B.4: Non-retrieval baseline system prompt (V0)

C Evaluation query set and assessment criteria

... Inhalt ...

D Additional results

... Inhalt ...

E Representative system outputs

... Inhalt ...